COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010/Data Structures and Algorithm	**SECTION:**	CPE21S3
DATE PERFORMED:	July 16, 2026	**DATE SUBMITTED:**	July 19, 2026
NAME:	Gresola, Aryanna Keith G.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 2.1 **Arrays, Pointers and Dynamic Allocation**

1. PROCEDURE OUTPUT

ILO A: Implement static and dynamic memory allocation & ILO B: Create dynamically allocated objects using pointers and arrays

1. INPUT

HOA_2P1MOD.cpp > ...

```cpp
#include <iostream>
#include <string.h>

class Student{
private:
    std::string studentName;
    int studentAge;

public:
    // Constructor
    Student(std::string newName = "Aryanna G", int newAge = 19){
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
    }

    // Destructor
    ~Student(){
        std::cout << "Destructor Called." << std::endl;
    }
```

```
21
22      // Copy Constructor
23      Student(const Student &copyStudent){
24          std::cout << "Copy Constructor Called." << std::endl;
25          studentName = copyStudent.studentName;
26          studentAge = copyStudent.studentAge;
27      }
28
29      // Display Attributes
30      void printDetails(){
31          std::cout << studentName << " " << studentAge << std::endl;
32      }
33  };
34
35  int main() {
36      Student student1("Roman", 28);
37      Student student2(student1);
38      Student student3;
39      student3 = student2;
40
41      return 0;
42  }
```

OUTPUT

```
[Running] cd "c:\Users\TIPQC\Downloads\GRESOLA\"
Constructor Called.
Copy Constructor Called.
Constructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
```

OBSERVATION

The program begins execution in the main() function. The first object, student1, is created, causing the program to transfer control to the constructor. Inside the constructor, the values passed as arguments are assigned to the object's private data members, studentName and studentAge. Once the constructor finishes initializing student1, control returns to the main() function. Next, student2 is created by copying student1, so instead of calling the regular constructor, the program invokes the copy constructor. The copy constructor receives student1 as its parameter and duplicates its studentName and studentAge into the newly created student2 object. After the copying process is complete, execution returns to main(). The program then creates student3, which is initialized with its own set of values, so the regular constructor is called again. The constructor assigns the new name and age to student3 before returning control to main(). Once all objects have been successfully created and initialized, the program proceeds to execute the class member function (such as printDetails()), which accesses each object's stored data and displays the corresponding name and age. After all statements inside main() have been executed, the program reaches its end. As the objects go out of scope, the destructor is automatically called for each object in the reverse order of their creation (student3, student2, then student1). Each destructor performs any necessary cleanup before the program terminates completely.

2. INPUT

```
#include <iostream>
#include <string.h>
class Student{
    private:
        std::string studentName;
        int studentAge;
    public:
        //constructor
        Student(std::string newName = "Aryanna G", int newAge = 19){
            studentName = std::move(newName);
            studentAge = newAge;
            std::cout << "Constructor Called." << std::endl;
        }

        //destructor
        ~Student(){
            std::cout << "Destructor Called." << std::endl;
        }
```

```
19
20          //copy constructor
21          Student(const Student &copyStudent){
22              std::cout << "Copy Constructor Called." << std::endl;
23              studentName = copyStudent.studentName;
24              studentAge = copyStudent.studentAge;
25          }
26
27          //display attributes
28          void printDetails(){
29              std::cout << this ->studentName << " " << this->studentAge << std::endl;
30          }
31
32  };
33
34  int main(){
35      const size_t j = 5;
36
37      Student studentList[j] = {};
38      std::string nameList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
39      int ageList[j] = {15, 16, 18, 19, 16};
40
41
42      return 0;
43  }
```

OUTPUT

```
[Running] cd "c:\Users\TIPQC\Downloads\GRESOLA\" && g++ HOA_2p1_MOD1.cpp -o
HOA_2p1_MOD1 && "c:\Users\TIPQC\Downloads\GRESOLA\"HOA_2p1_MOD1
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
```

OBSERVATION

The code starts in the main() function, where it first sets the constant size j to 5. It then creates the studentList array, which contains five Student objects. Since no values are provided during their creation, the constructor uses its default parameters, assigning the name "Aryanna G" and the age 18 to each object. After initializing each object, the message "Constructor Called." is displayed. This process repeats five times because there are five Student objects in the array. Once all objects have been constructed, the program returns to the main() function and creates the nameList array, storing the names Carly, Freddy, Sam, Zack, and Cody, followed by the ageList array containing the values 15, 16, 18, 19, and 16. These two arrays are created independently and are not assigned to the Student objects. After all statements in main() have finished executing, the program reaches return 0, signaling the end of the program. Before the program terminates, local variables are automatically destroyed in reverse order of their creation. The ageList and nameList arrays are destroyed first without producing any output, and then the studentList array is destroyed. As each Student object is destroyed, its destructor is called, displaying "Destructor Called." five times. During this process, the memory occupied by each Student object's studentName and studentAge is released until all five objects have been destroyed and the program ends.

3. INPUT

```cpp
#include <iostream>
#include <string.h>

class Student{
private:
    std::string studentName;
    int studentAge;

public:
    // Constructor
    Student(std::string newName = "Aryanna G", int newAge = 19){
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
    }

    // Destructor
    ~Student(){
        std::cout << "Destructor Called." << std::endl;
    }
```

```
21
22      // Copy Constructor
23      Student(const Student &copyStudent){
24          std::cout << "Copy Constructor Called." << std::endl;
25          studentName = copyStudent.studentName;
26          studentAge = copyStudent.studentAge;
27      }
28
29      // Display Attributes
30      void printDetails(){
31          std::cout << studentName << " " << studentAge << std::endl;
32      }
33  };
34
35  int main() {
36      const size_t j = 5;
37
38      Student studentList[j] = {};
39      std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
40      int ageList[j] = {15, 16, 18, 19, 16};
41
42      for(int i = 0; i < j; i++){ //loop A
43          Student *ptr = new Student(namesList[i], ageList[i]);
44          studentList[i] = *ptr;
45  }
46
47  for(int i = 0; i < j; i++){ //loop B
48      studentList[i].printDetails();
49      }
50      return 0;
51  }
```

LOOP A

```
42      for(int i = 0; i < j; i++){ //loop A
43          Student *ptr = new Student(namesList[i], ageList[i]);
44          studentList[i] = *ptr;
45  }
```

OBSERVATION

Loop A runs 5 times because the constant j is set to 5. In every iteration, the program creates a new Student object using the current values from namesList and ageList. This calls the constructor again, so the message "Constructor Called." is printed every time. After that, the program copies the new student's name and age into the corresponding object in studentList, replacing the default values that were assigned earlier. The copy constructor is not called because the object already exists and is only being assigned new values. Since the dynamically created objects are not deleted after they are used, they remain in memory, which results in a memory leak.

LOOP B

```
47  for(int i = 0; i < j; i++){ //loop B
48      studentList[i].printDetails();
49      }
50      return 0;
51  }
```

OBSERVATION

Loop B also runs 5 times because the constant j is set to 5. In each iteration, the program goes through the updated studentList array and calls the printDetails() function for each Student object. The function retrieves the student's current name and age and displays them on the screen. Since studentList was already updated in Loop A, the output shows the new names and ages instead of the default values. After the loop finishes printing all five student records, the program continues until it reaches the end of the main() function, where the destructors are called as the studentList objects are destroyed.

OUTPUT

```
[Running] cd "c:\Users\TIPQC\Downloads\GRESOLA\" && g++ HOA_2p1_MOD2.cpp -o
HOA_2p1_MOD2 && "c:\Users\TIPQC\Downloads\GRESOLA\"HOA_2p1_MOD2
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Carly 15
Freddy 16
Sam 18
Zack 19
Cody 16
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
```

OBSERVATION

The program begins execution in the main() function by setting the constant j to 5, which determines the size of all the arrays. It first creates the studentList array containing five Student objects. Since no values are provided during their creation, each object uses the default constructor, assigning the default name "Aryanna G" and age 19. As each object is created, the message "Constructor Called." is displayed, resulting in five constructor calls. The program then initializes the namesList and ageList arrays with the names and ages that will be assigned to the students

The program then proceeds to Loop A, which iterates five times. In each iteration, a new Student object is dynamically created using the corresponding values from namesList and ageList, causing the constructor to be called another five times. The data from this newly created object is then assigned to the corresponding element in studentList, replacing the default name and age with the new values. The copy constructor is not called because the objects in studentList already exist and are simply being assigned new values.

After all the student objects have been updated, the program enters Loop B, which also runs five times. During each iteration, the printDetails() function is called to display the student's updated name and age. Finally,

when the main() function finishes, the studentList array goes out of scope, causing the destructor to be called five times as each object is destroyed. However, the dynamically allocated objects created with new inside Loop A are never deleted, so their destructors are not executed. This leaves those objects in memory, creating a memory leak. Therefore, the total execution consists of 10 constructor calls (5 default constructors for studentList and 5 parameterized constructors inside Loop A), 0 copy constructor calls, and 5 destructor calls before the program completely terminates.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Problem 1: Create a class for the fruit and the vegetable classes. Each class must have a constructor, deconstructor, copy constructor and copy assignment operator. They must also have all relevant attributes (such as name, price and quantity) and functions (such as calculate sum) as presented in the problem description above.

INPUT

Supp_HOA2P1.cpp > GroceryItem(const GroceryItem &)

```cpp
#include <iostream>
#include <string>

class GroceryItem{
protected:
    std::string name;
    double price;
    int quantity;

public:
    GroceryItem();
    GroceryItem(std::string, double, int);
    GroceryItem(const GroceryItem&);
    GroceryItem& operator=(const GroceryItem&);
    virtual ~GroceryItem();

    virtual double calculateSum() const = 0;
    virtual void displayItem() const = 0;
};
```

```cpp

// grocery definition
GroceryItem::GroceryItem(){
    name="";
    price=0;
    quantity=0;
}

GroceryItem::GroceryItem(std::string n,double p,int q){
    name=n;
    price=p;
    quantity=q;
}

GroceryItem::GroceryItem(const GroceryItem& other){
    name=other.name;
    price=other.price;
    quantity=other.quantity;
}

GroceryItem& GroceryItem::operator=(const GroceryItem& other){
    if(this!=&other){
        name=other.name;
        price=other.price;
        quantity=other.quantity;
    }
    return *this;
}

GroceryItem::~GroceryItem(){}

```

```cpp
// fruit class

class Fruit : public GroceryItem{
public:
    // constructor
    Fruit();
    Fruit(std::string, double, int q);
    Fruit(const Fruit&); // copy constructor
    Fruit& operator=(const Fruit&); // copy assignment operator
    ~Fruit() override; // destructor

    // getters
    std::string getName() const;
    double getPrice() const;
    int getQuantity() const;

    // setters
    void setPrice(double);
    void setQuantity(int);

    double calculateSum() const override;
    double calculateDiscount(double percent) const;
    bool isAvailable() const;
    void updateItem(std::string, double, int);
    void displayItem () const override;
};

// function definition
// Fruit:: - tells compiler fruit belongs to fruit class
// != &other - prevents assigning an object to itself in the copy asgn op
// return *this - returns the current obj
```

```cpp

Fruit::Fruit() : GroceryItem() {} // constructor

//parametized constructor
Fruit::Fruit(std::string n, double p, int q) : GroceryItem(n,p,q) {}

//copy constructor
Fruit::Fruit(const Fruit& other) : GroceryItem(other) {}

// copy assignment operator
Fruit& Fruit::operator=(const Fruit& other){
    if (this != &other){
        GroceryItem::operator=(other);
    }
    return *this;
}
```

```cpp

Fruit::~Fruit() {}

// getters
std::string Fruit::getName() const{
    return name;
}
double Fruit::getPrice() const{
    return price;
}
int Fruit::getQuantity() const{
    return quantity;
}

//setters
void Fruit::setPrice(double p){
    price = p;
}
void Fruit::setQuantity(int q){
    quantity = q;
}

// other funcs:
double Fruit::calculateSum() const{ // total price
    return price * quantity;
}

double Fruit::calculateDiscount(double percent) const{
    return calculateSum() * (1 - percent / 100); // discount
}

bool Fruit::isAvailable() const{ // availability
    return quantity > 0;
}

void Fruit::updateItem(std::string n, double p, int q){
    name = n;
    price = p;
    quantity = q; // updates item detail
}

void Fruit::displayItem() const{
    std::cout << "Fruit" << std::endl;
    std::cout << "Name: " << name << std::endl;
    std::cout << "Price: PHP " << price << std::endl;
    std::cout << "Quantity: " << quantity << std::endl;
    std::cout << "Total: PHP " << calculateSum() << std::endl;
}
```

```cpp

class Vegetable : public GroceryItem{
public:
    // Constructors
    Vegetable();
    Vegetable(std::string, double, int);
    Vegetable(const Vegetable&);              // Copy Constructor
    Vegetable& operator=(const Vegetable&);   // Copy Assignment Operator
    ~Vegetable() override;                        // Destructor

    // Getters
    std::string getName() const;
    double getPrice() const;
    int getQuantity() const;

    // Setters
    void setPrice(double);
    void setQuantity(int);

    // Other Functions
    double calculateSum() const override;
    double calculateDiscount(double percent) const;
    bool isAvailable() const;
    void updateItem(std::string, double, int);
    void displayItem() const override;
};

// vegetable funcs
Vegetable::Vegetable() : GroceryItem() {}

Vegetable::Vegetable(std::string n, double p, int q) : GroceryItem(n,p,q) {}

Vegetable::Vegetable(const Vegetable& other) : GroceryItem(other) {}

Vegetable& Vegetable::operator=(const Vegetable& other){
    if (this != &other){
        GroceryItem::operator=(other);
    }
    return *this;
}
```

```cpp

Vegetable::~Vegetable() {}

// getters
std::string Vegetable::getName() const{
    return name;
}
double Vegetable::getPrice() const{
    return price;
}
int Vegetable::getQuantity() const{
    return quantity;
}

// setters
void Vegetable::setPrice(double p){
    price = p;
}
void Vegetable::setQuantity(int q){
    quantity = q;
}

// other funs: vegies
double Vegetable::calculateSum() const{
    return price * quantity;
}

// Computes the total after discount
double Vegetable::calculateDiscount(double percent) const{
    return calculateSum() * (1 - percent / 100);
}

// Checks if the item is available
bool Vegetable::isAvailable() const{
    return quantity > 0;
}

// Updates all item information
void Vegetable::updateItem(std::string n, double p, int q){
    name = n;
    price = p;
    quantity = q;
}

// Displays the item details
void Vegetable::displayItem() const{
    std::cout << "Vegetables" << std::endl;
    std::cout << "Name: " << name << std::endl;
    std::cout << "Price: PHP " << price << std::endl;
    std::cout << "Quantity: " << quantity << std::endl;
    std::cout << "Total: PHP " << calculateSum() << std::endl;
}
```

Problem 2: Create an array GroceryList in the driver code that will contain all items in Jenna's Grocery List. You must then access each saved instance and display all details about the items.

INPUT

```
252  int main(){
253
254      // array of pointers (dynamic memory allocation) problem 2
255      GroceryItem* GroceryList[4];
256
257      // allocate memory using 'new'
258      GroceryList[0] = new Fruit("Apple",10,7);
259      GroceryList[1] = new Fruit("Banana",10,8);
260      GroceryList[2] = new Vegetable("Broccoli",60,12);
261      GroceryList[3] = new Vegetable("Lettuce",50,10);
262
263      std::cout << "JENNA'S GROCERY LIST" << std::endl << std::endl;
264      // display all
265      for(int i = 0; i < 4; i++){
266          GroceryList[i]->displayItem(); // uses -> because GroceryList[i] is a ptr
267          std::cout << std::endl;
268      }
```

OUTPUT

```
Fruit
Name: Apple
Price: PHP 10
Quantity: 7
Total: PHP 70

Fruit
Name: Banana
Price: PHP 10
Quantity: 8
Total: PHP 80
```

```
Vegetables
Name: Broccoli
Price: PHP 60
Quantity: 12
Total: PHP 720

Vegetables
Name: Lettuce
Price: PHP 50
Quantity: 10
Total: PHP 500
```

Problem 3: Create a function TotalSum that will calculate the sum of all objects listed in Jenna's Grocery List.

INPUT

- Outside the main function:

```
240  double TotalSum(GroceryItem* GroceryList[], int size){ // problem 3
241      double total = 0;
242
243      // adds price for all
244      for(int i = 0; i < size; i++){
245          if(GroceryList[i] != nullptr){
246              total += GroceryList[i]->calculateSum();
247          }
248      }
249      return total;
250  }
```

- Calling it inside the main function:

```
270      std::cout << "----------------------" << std::endl; // problem 3
271      std::cout << "Total Grocery Cost: PHP " << TotalSum(GroceryList, 4) << std::endl
272                << std::endl;
273
```

OUTPUT

```
----------------------
Total Grocery Cost: PHP 1370
```

Problem 4: Delete the Lettuce from Jenna's GroceryList list and de-allocate the memory assigned.

INPUT

```
274     // problem 4
275     std::cout << "Deleting Lettuce..." << std::endl;
276
277     delete GroceryList[3];
278     GroceryList[3] = nullptr;
279
280     std::cout << "Lettuce has been removed from the grocery list." << std::endl << std::endl;
281
282     std::cout << "UPDATED GROCERY LIST: " << std::endl;
283     for(int i = 0; i < 4; i++){
284         if(GroceryList[i] != nullptr){
285             GroceryList[i]->displayItem();
286             std::cout << std::endl;
287         }
288     }
```

OUTPUT

```
Deleting Lettuce...
Lettuce has been removed from the grocery list.

UPDATED GROCERY LIST:
Fruit
Name: Apple
Price: PHP 10
Quantity: 7
Total: PHP 70

Fruit
Name: Banana
Price: PHP 10
Quantity: 8
Total: PHP 80

Vegetables
Name: Broccoli
Price: PHP 60
Quantity: 12
Total: PHP 720
```

OVERALL OUTPUT

```
Fruit
Name: Apple
Price: PHP 10
Quantity: 7
Total: PHP 70

Fruit
Name: Banana
Price: PHP 10
Quantity: 8
Total: PHP 80

Vegetables
Name: Broccoli
Price: PHP 60
Quantity: 12
Total: PHP 720

Vegetables
Name: Lettuce
Price: PHP 50
Quantity: 10
Total: PHP 500

--------------------
Total Grocery Cost: PHP 1370
```

```
Deleting Lettuce...
Lettuce has been removed from the grocery list.

UPDATED GROCERY LIST:
Fruit
Name: Apple
Price: PHP 10
Quantity: 7
Total: PHP 70

Fruit
Name: Banana
Price: PHP 10
Quantity: 8
Total: PHP 80

Vegetables
Name: Broccoli
Price: PHP 60
Quantity: 12
Total: PHP 720
```

CODE ANALYSIS/OBSERVATION

This program manages Jenna's grocery list by creating grocery items, displaying their details, calculating the total cost, and updating the list after removing an item. It starts by defining a GroceryItem class that contains the common attributes name, price, and quantity, along with the constructors, copy constructor, copy assignment operator, and destructor.

The default constructor initializes the attributes with default values, while the parameterized constructor is used to create grocery items with specific values. The copy constructor and copy assignment operator are also included to correctly copy an object's data whenever needed, even though they are not directly used in the program. The destructor is automatically called when an object is deleted, allowing the dynamically allocated memory to be released properly.

In the main() function, the grocery items are created using dynamic memory allocation and stored in the GroceryList array. Using an array makes it easier to loop through all items, display their information, and calculate the total grocery cost through the TotalSum() function. Finally, the program deletes the Lettuce object, sets its pointer to nullptr, displays the updated grocery list, and deallocates the remaining objects before the program ends. Overall, this implementation helped me better understand how constructors, memory management, and arrays work together when handling multiple objects in C++.

LEARNINGS

Through this activity, I learned how constructors are used to initialize objects when they are created, while the copy constructor and copy assignment operator are used to correctly copy the data of one object to another. I also learned that a destructor is automatically called when an object is deleted, making it important for properly releasing dynamically allocated memory and preventing memory leaks.

I also gained a better understanding of dynamic memory allocation using the new and delete operators. Using new creates objects during program execution, while delete deallocates the memory once the object is no longer needed. This helped me understand why setting a pointer to nullptr after deletion is considered good practice.

Another important concept I learned is using an array of pointers to store multiple objects. Instead of creating separate variables for every item, an array makes it easier to access, display, update, and process all objects using loops. I also learned how passing an array to a function, such as TotalSum(), allows the program to process multiple objects efficiently.

Overall, this activity strengthened my understanding of object creation, memory management, arrays, and functions. These are important foundations that I can apply in future lessons, especially in this course where efficient data storage, memory handling, and manipulating collections of objects become more important.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: ChatGPT

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Code assistance, explanation of concepts, and grammar correction.

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: ChatGPT was used to gain a better understanding of the code by explaining C++ concepts, code flow, and programming errors. It was also used to suggest debugging techniques, identify possible improvements to the code, and assist with grammar correction. It did not replace my original thinking, analysis, authorship, or completion of the required work.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.